

Herald



Herald — Messenger Channels (Operator Guide)

Field	Value
Revision	2
Created	2026-05-28
Last modified	2026-05-31
Status	active
Status summary	r2: added §6A (Participant identity, attribution & @-tagging) documenting the participant/attribution contract (<code>docs/design/PARTICIPANT_ATTRIBUTION.md</code>) — per-channel subscribers/<code>subscriber_aliases.username</code> mapping, the <code>HERALD_<CHANNEL>_OPERATOR_USERNAME</code> operator env var (<code>HERALD_TGRAM_OPERATOR_USERNAME=@milos85vasic</code>), and the @-tagging matrix (Claude + Operator never tagged). Operator-facing reference for Herald's Wave 7 generic messenger-channel framework. Documents the <code>commons_messaging/channels.Channel</code> interface (the richer inbound contract that embeds the §11.0 outbound <code>commons.Channel</code>), the <code>init()</code>-driven registry that resolves channel names at runtime, the per-channel content-addressed inbox (<code>~/herald/inbox/<channel>/<sha256>.<ext></code>), the generalized self-filter that defends every adapter against echo-loops, the <code>HERALD_CHANNELS</code> selector that drives multi-channel <code>pherald listen</code>, the two live adapters (Telegram since Wave 6, Slack since Wave 7 T6), the URL-scheme parsers, the implementer checklist for adding a new channel, the troubleshooting cookbook, and the operator pre-deploy audit checklist.
Issues	none
Issues summary	—
Fixed	(n/a — new guide)
Continuation	bump when new channels land (Wave 8 Max/Email per <code>docs/CONTINUATION.md §3</code>); bump when the channels registry gains hot-reload or per-channel rate-limit knobs; bump when a <code>pherald listen --dry-run</code> (registry-resolution probe) ships.

Table of contents

- [§1. Architecture overview](#)
 - [§2. Enabling channels \(HERALD_CHANNELS\)](#)
 - [§3. Telegram \(LIVE — Wave 6 HRD-011\)](#)
 - [§4. Slack \(LIVE — Wave 7 T6 HRD-115\)](#)
 - [§5. Per-channel inbox isolation](#)
 - [§6. Self-filter \(anti-echo-loop\)](#)
 - [§6A. Participant identity, attribution & @-tagging](#)
 - [§7. URL schemes](#)
 - [§8. Adding a new channel](#)
 - [§9. Troubleshooting](#)
 - [§10. Audit checklist \(operator pre-deploy\)](#)
 - [§11. References](#)
-

§1. Architecture overview

§1.1 Two interfaces, one contract

Herald's messenger plane has two complementary contracts. The outbound contract is the **spec §11.0 commons.Channel** — the five-method shape every flavor's `serve` plane depends on (`Name`, `Capabilities`, `Send`, `Subscribe`, `HealthCheck`). Widening that interface would ripple into every flavor's outbound path, so Wave 7 introduced a strictly richer inbound contract in a `NEW` package without touching the existing one:

Layer	Package	Methods	Consumers
Outbound contract (§11.0)	<code>commons</code>	<code>Name</code> , <code>Capabilities</code> , <code>Send</code> , <code>Subscribe</code> , <code>HealthCheck</code>	every flavor's <code>serve</code> plane
Inbound contract (Wave 7)	<code>commons_messaging/channels</code>	the five above (embedded) plus <code>SendReplyGeneric</code> , <code>BotSelfIdentity</code> , <code>DownloadAttachment</code>	<code>pherald</code> <code>listen</code> , <code>qaherald</code>

The richer interface lives at `commons_messaging/channels/channel.go`. Every concrete adapter (Telegram, Slack, future Max/Email/Discord) satisfies it; the inbound runtime depends **ONLY** on the interface, never on the concrete adapter type, so adding a new channel does **NOT** touch `pherald listen`'s hot path.

§1.2 The eight-method shape (full)

```
type Channel interface {  
    commons.Channel // 5  
    outbound methods (§11.0)  
    SendReplyGeneric( //  
        quoted reply with attachments  
        ctx context.Context,  
        recipient commons.Recipient,  
        body string,  
        replyToID string,
```

```

        attachments []commons.Attachment,
    ) (string, error)
    BotSelfIdentity(ctx context.Context) (SelfIdentity,
        error) // anti-echo-loop
    DownloadAttachment( //
        content-addressed inbox write
        ctx context.Context,
        externalID string,
        mime string,
    ) (string, string, error) //
        (finalPath, sha256Hex, err)
}

```

A `SelfIdentity` is a `{Kind, Value}` pair where `Kind` is one of `IdentityUsername` (Telegram bot.Me.Username), `IdentityUserID` (Slack bot_user_id), or `IdentityAddress` (Email From-address — Wave 8 reserved).

§1.3 Reply-method naming rationale (load-bearing footnote)

`SendReplyGeneric` exists rather than `SendReply` because `*tgram.Adapter` predates the interface and already exposes a native `SendReply(ctx, chatID int64, replyToMessageID int, ...)`. A Go type may hold only one method named `SendReply`; the only way to preserve the existing `tgram` behaviour while adding the channel-agnostic method was to give the new method a different name. The matching `inbound.Replier` interface (`pherald/internal/inbound`) uses the shorter `SendReply` because it is freshly introduced — and is bridged to `Channel.SendReplyGeneric` by the thin `channelReplier` shim in `pherald/cmd/pherald/listen.go`.

§1.4 The registry — name → constructor

`commons_messaging/channels/registry.go` is a tiny package-global `map[string]Constructor` populated by each adapter's `init()`. The registry is the entire abstraction between the adapter implementation and the runtime: `pherald listen blank-` imports the adapter packages, the adapters self-register, and `channels.New("tgram", cfg)` (or "slack", etc.) resolves by name.

```

// commons_messaging/channels/tgram/tgram.go
func init() {
    channels.Register(string(common.ChannelTelegram), func(cfg
        channels.Config) (channels.Channel, error) {
        if cfg.BaseURL != "" {
            return NewAdapterWithBaseURL(cfg.Token, cfg.Target,
                cfg.BaseURL), nil
        }
        return NewWithCreds(cfg.Token, cfg.Target), nil
    })
}

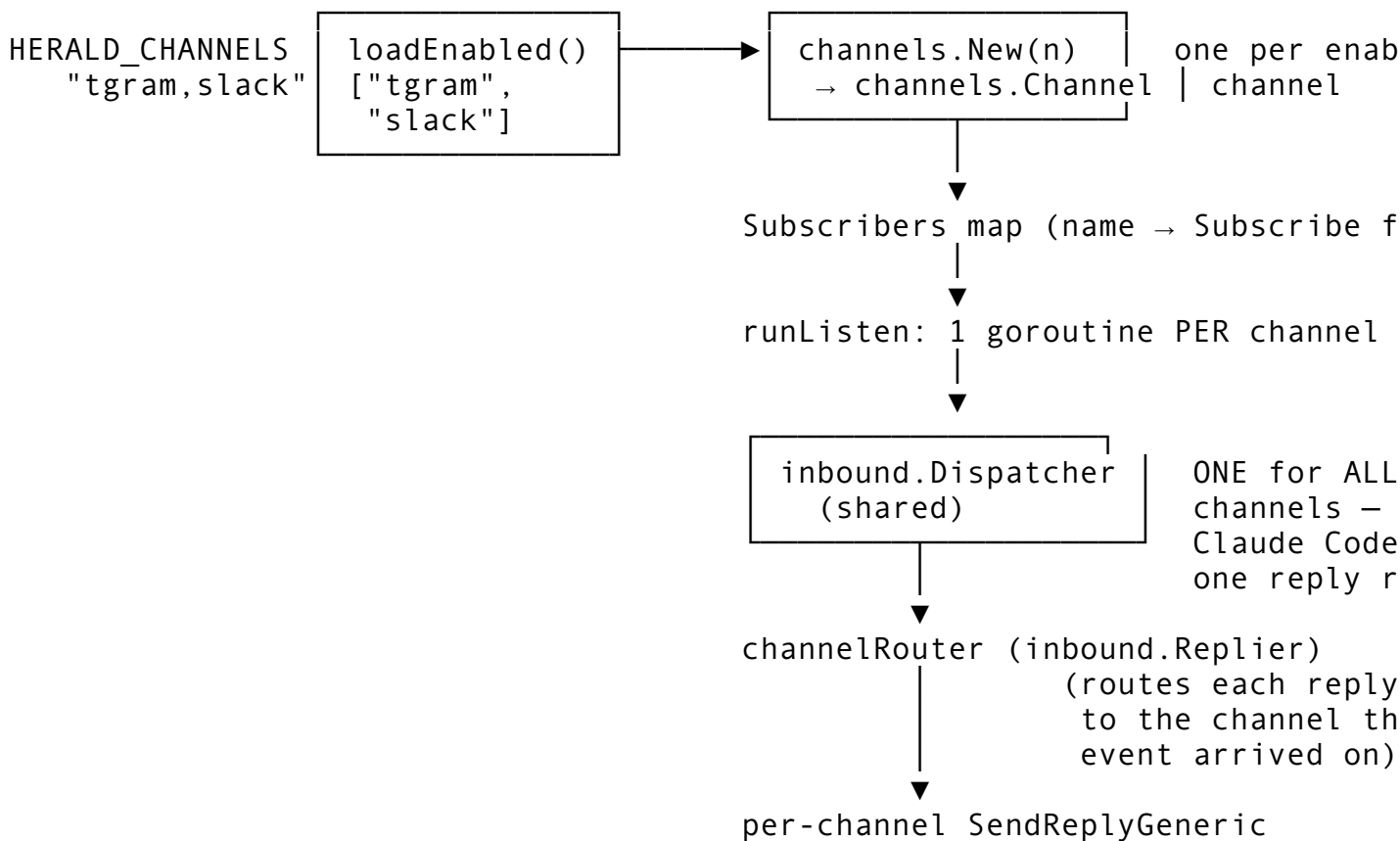
```

Duplicate registration **panics** at process start — a typo that shadowed one of two adapters with the same name would silently route traffic to the wrong adapter (a classic §107 PASS-bluff), so the registry refuses to load.

`channels.Names()` returns the sorted list of registered names;
`channels.New(unknown, cfg)` returns `ErrUnknownChannel` with the registered set embedded in the error message (fail-loud — no silent no-op channel).

§1.5 The runtime — pherald listen fan-in

`pherald listen` (`pherald/cmd/pherald/listen.go`) wires the registry to the inbound dispatcher:



The fan-in semantics are deliberate: ONE Claude Code session sees subscriber traffic from EVERY enabled channel, and the dispatcher's `channelRouter` (a small `map[string]inbound.Replier` keyed by channel name) routes each reply back to the adapter the inbound event arrived on. A Slack message gets a Slack reply; a Telegram message gets a Telegram reply; there is no cross-channel reply leak. If any subscriber goroutine returns a non-cancellation error, `runListen` cancels its siblings and propagates the error — silent partial failure is forbidden (`pherald/cmd/pherald/listen.go:399..462`).

§2. Enabling channels (HERALD_CHANNELS)

`HERALD_CHANNELS` is the single env var that controls which channels Wave 7 `pherald listen` brings up:

Value	Behaviour
<code>unset</code> or empty	Wave 6 single-channel default — Telegram only (<code>tgram</code>).
<code>tgram</code>	Telegram only. Identical to the default.
<code>tgram, slack</code>	Both Telegram and Slack — one Subscribe goroutine per channel, one shared inbound dispatcher.
<code>slack</code>	Slack only (Telegram disabled — no <code>HERALD_TGRAM_*</code> required).
<code>tgram, foo</code>	FAILS LOUD AT BOOT — <code>foo</code> is not registered, so <code>channels.New("foo")</code> returns <code>ErrUnknownChannel</code> .

Whitespace inside the value is tolerated; ordering does not matter; duplicates are collapsed; leading/trailing/double commas are skipped. The parser is `loadEnabledChannels()` (`pherald/cmd/pherald/listen.go:289..305`).

Fail-loud invariants.

1. Unknown channel name → `pherald listen` refuses to boot with `ErrUnknownChannel: "foo" (registered: [slack tgram])`.
2. An enabled channel whose required env is unset → boot fails with a per-channel descriptive error (e.g. `HERALD_SLACK_BOT_TOKEN required (channel slack enabled)`).
3. Empty resolved channel set (a refactor regression) → boot fails with `no channels enabled (HERALD_CHANNELS resolved empty)`.
4. Any subscriber goroutine that returns a non-cancellation error after boot → `runListen` cancels every sibling subscriber and returns the error to the caller. Silent partial degradation is impossible by construction.

No `--allow-unknown-channels / --skip-credential-check / --single-channel-fallback` escape hatch exists; per §107 anti-bluff, a silently-degraded listener is worse than a hard failure because it claims to work for end users.

§3. Telegram (LIVE — Wave 6 HRD-011)

§3.1 Required environment

Variable	Required for	Format	Example
<code>HERALD_TGRAM_BOT_TOKEN</code>	outbound + inbound	<bot-id>:<api-token> from BotFather numeric (group:	1234567890:AAH1lab2c3d4e5f6g7h8i9j
<code>HERALD_TGRAM_CHAT_ID</code>	outbound default destination	negative, supergroup: starts -100, DM: positive)	-1001234567890

§3.2 Obtaining a bot token

1. Open Telegram and DM @BotFather.
2. Send /newbot. Pick a display name (any string) and a username ending in bot (must be globally unique).
3. BotFather replies with Use this token to access the HTTP API: followed by your token. Copy it into HERALD_TGRAM_BOT_TOKEN.
4. (Optional — strongly recommended for groups.) Send /setprivacy to BotFather, select your bot, choose Disable. Without this, group members' messages are NOT delivered to your bot via getUpdates. Privacy-mode-on is the single most common cause of "the bot doesn't see my messages" tickets.

§3.3 Obtaining a chat id

For a 1:1 DM:

1. Send any message to your bot.
2. Curl `https://api.telegram.org/bot<TOKEN>/getUpdates` and read the `result[].message.chat.id` field — that integer is HERALD_TGRAM_CHAT_ID.

For a group / supergroup:

1. Add the bot to the group, send a message.
2. Curl `getUpdates` as above. Group chat ids are negative integers; supergroup chat ids start with -100.

§3.4 Transport (under the hood)

- **Outbound.** `sendMessage` (text), `sendPhoto` / `sendDocument` / `sendVoice` (attachments). Markdown rendering uses Telegram's MarkdownV2 dialect.
- **Inbound.** `getUpdates` long-poll with `timeout=25s`. Each poll cycle drains all pending updates, hands them to `OnPhoto` / `OnDocument` / `OnVoice` / generic message handlers, and re-polls.
- **Self-filter.** `bot.Me.Username` is read once at Subscribe construction (cached for the process lifetime) — see `commons_messaging/channels/tgram/subscribe.go`.
- **Reply threading.** `reply_to_message_id` quotes the originating message. Forum-topic threading via `message_thread_id` is supported in V2.
- **Attachments.** Photos / documents / voice messages are streamed via `getFile` + the file-download endpoint into `~/herald/inbox/tgram/<sha256>.<ext>` (see §5).

§3.5 Capabilities

Text	: true
Markdown	: true (MarkdownV2 dialect)
HTML	: true
Attachments	: true
AttachmentMaxMiB	: 50 (Telegram Bot API hard limit)
Threads	: true (forum topics via <code>message_thread_id</code>)
InteractiveURL	: true
InteractiveCall	: false (V2 advanced — HRD-011 follow-up)
DeliveryCeiling	: DeliveryRouted (200 OK == platform stored)

§3.6 Reading the diary

Every message in and out is appended to `docs/herald/diary/main.md` (and re-exported to `main.pdf/main.html` per the conversation-diary invariant — see `CLAUDE.md`).

§4. Slack (LIVE — Wave 7 T6 HRD-115)

§4.1 Required environment

Variable	Required for	Format	Example
HERALD_SLACK_BOT_TOKEN	outbound + identity	starts xoxb-	xoxb-1234-5678-AbCdEfGh
HERALD_SLACK_APP_TOKEN	inbound (Socket Mode)	starts xapp-	xapp-1-A0123-456-XYZ
HERALD_SLACK_CHANNEL_ID	outbound default destination	starts C (public) / G (private)	C0123456789

HERALD_SLACK_APP_TOKEN is required ONLY when the Slack channel participates in inbound (i.e. when HERALD_CHANNELS includes `slack`). An outbound-only Slack deployment can omit it.

§4.2 Creating a Slack app

1. Visit <https://api.slack.com/apps> and click **Create New App** → **From scratch**.
2. Name the app, pick the target workspace, click **Create**.
3. In the app's settings, navigate to **OAuth & Permissions** → **Scopes** → **Bot Token Scopes** and add:
 - `chat:write` — required for `chat.postMessage`.
 - `files:read` — required for inbound attachment retrieval.
 - `files:write` — required for outbound attachment upload (`files.uploadV2`).
 - `channels:history` — required for public-channel inbound. Use `groups:history` instead if the bot will live in private channels.
 - `users:read` — recommended, lets Herald enrich the sender display name in the diary.
4. Navigate to **Socket Mode** and toggle **Enable Socket Mode**. Slack then prompts you to generate an app-level token with the `connections:write` scope — generate it and copy the `xapp-...` value into `HERALD_SLACK_APP_TOKEN`.
5. Navigate to **Event Subscriptions**, toggle **Enable Events**, then subscribe the bot to (at minimum) `message.channels` (public) and/or `message.groups` (private). Socket Mode means there is NO public webhook URL to configure — events arrive over the WebSocket.
6. Navigate to **Install App** → **Install to Workspace**. Authorize the install; Slack returns the **Bot User OAuth Token** (`xoxb-...`) — copy it into `HERALD_SLACK_BOT_TOKEN`.
7. Add the bot to the target channel: in the Slack client run `/invite @your-bot-name`. Then right-click the channel name → **View channel details** → copy the channel id (`C...` or `G...`) into `HERALD_SLACK_CHANNEL_ID`.

§4.3 Transport (under the hood)

- **Outbound.** `chat.postMessage` for text and Slack-flavored markdown (`mrkdwn`); `files.uploadV2` for attachments.
- **Inbound.** Socket Mode — a persistent WebSocket originated by Herald to Slack's edge. No inbound NAT traversal, no public URL, no webhook signature verification — Slack's onus, not the operator's. This is the equivalent of Telegram's `getUpdates` long-poll.
- **Self-filter.** `auth.test` is called once at Subscribe construction. The returned `bot_user_id` (a U... value such as `U01ABC`) is cached for the process lifetime; an empty result is fail-loud (see §6).
- **Reply threading.** Replies use `thread_ts` — Slack's anchor for reply threading. The `channelRouter` passes the originating message's `ts` through to `SendReplyGeneric` as `replyToID`, the Slack adapter rewrites it as `thread_ts`.
- **Attachments.** Inbound attachments arrive as `file_share` subtype events; the adapter follows `file.url_private_download` (presigned, requires the bot token in the `Authorization: Bearer ...` header) and streams the bytes into `~/ .herald/ inbox/slack/<sha256>.<ext>` (see §5).

§4.4 Capabilities

Text	: true	
Markdown	: true	(Slack mrkdwn dialect — <i>*bold*</i> , <i>_italic_</i> , <code>`code`</code>)
HTML	: false	(Slack rejects HTML — adapters render mrkdwn)
Attachments	: true	
AttachmentMaxMiB	: 1024	(workspace-plan dependent; this is the API so)
Threads	: true	(<code>thread_ts</code>)
InteractiveURL	: true	
InteractiveCall	: false	(interactive Block Kit / call routing — futur)
DeliveryCeiling	: DeliveryRouted	(<code>postMessage 200 == platform stored</code>)

§4.5 §107 security note — token redaction

`xoxb-...` tokens are workspace-scope credentials that, if leaked into a log line or error message, give the holder full bot privileges. The Slack adapter and the `qaherald` Slack scenario BOTH carry a `sanitizeError` step that scrubs `xoxb- / xapp-` substrings from every error returned to the caller. The invariant is pinned by `TestSlack_Send_ErrorDoesNotLeakToken` (`commons_messaging/channels/slack/send_test.go`) — a mutation that disabled the sanitizer would surface immediately. Operators should still avoid `set -x` in production wrappers and should rotate any token that appears in a captured trace.

§5. Per-channel inbox isolation

Every inbound attachment is content-addressed: the sha256 of the bytes IS the filename stem; the MIME-derived extension is the only suffix. The directory is namespaced by channel so a Telegram photo and a Slack photo with identical bytes do NOT collide and the on-disk forensic trail is self-describing:

```
~/ .herald/ inbox/
├── tgram/
│   ├── 6a8d2c1f... .jpg      (Telegram photo)
│   └── 9b3e4d5f... .ogg      (Telegram voice)
```



```

└─ slack/
   └─ 6a8d2c1f...jpg      (Slack file with byte-identical content — coexis
   └─ 7c8a9b0d...pdf      (Slack document)
   ...

```

Override the inbox root via `HERALD_INBOX_DIR` (the parent of the per-channel subdirs); the per-channel subdir naming is fixed and matches the channel's registered name. Permissions are `0700` on the per-channel directory (no other-user read).

The write helper is `channels.WriteContentAddressed(channel, mime, body)` (`commons_messaging/channels/inbox.go`). It uses `io.MultiWriter(tempFile, sha256Hasher)` to hash inline as bytes stream to disk — no full-payload buffering — then atomic-renames into the final content-addressed path. Idempotence is built in: if the content-addressed final path already exists (a duplicate download), the temp file is discarded and the existing path is returned unchanged. The corresponding tests (`inbox_test.go` + adapter-level `attachments_test.go`) pin the exact path shape, byte-equality, and absence of `.part` residue.

§6. Self-filter (anti-echo-loop)

Every Herald channel adapter is a bot that BOTH sends and receives in the same conversation. Without active filtering, every reply the dispatcher emits would arrive back as a fresh subscriber message, triggering another Claude Code dispatch, triggering another reply — an exponential loop that would saturate the long-poll within the 25-second timeout on the first reply. Wave 6 defended this in `tgram` via `bot.Me.Username`; Wave 7 generalized the defence to a channel-agnostic `BotSelfIdentity + IsSelfEcho` pair.

§6.1 The flow

1. **Adapter construction time:** `Subscribe` is called. Before returning, the adapter calls its own `BotSelfIdentity(ctx)` (the channel's native identity API — Telegram's `getMe`, Slack's `auth.test`, future Email's From-address resolver). The result is cached.
2. **Boot fail-loud:** if the identity result is empty (`SelfIdentity.Value == ""`), `Subscribe` refuses to start. An echo-defenceless listener is forbidden by construction — there is no `--allow-empty-identity` knob.
3. **Per-message:** the adapter's inbound handler builds an `InboundEvent` and calls `channels.StampSender(ev.Raw, isBot, kind, value)` to stamp the sender's native identity into the event's raw map.
4. **Filter time:** `channels.IsSelfEcho(ev, self)` compares the stamped sender identity against the cached self-identity — same `Kind`, same `Value`, sender flagged as bot → echo, drop. Different bot (or human user) → keep, dispatch to Claude Code.

§6.2 The comparator

`IsSelfEcho` (`commons_messaging/channels/selffilter.go`) is deliberately narrow:

- Different-bot messages are KEPT — multi-bot collaboration in the same conversation is real subscriber traffic and must not be silently dropped.
- Empty self-identity is treated as a conservative KEEP — but `Subscribe` should have already refused to boot in that case, so reaching the filter with empty self is a defence-in-depth

fallback that surfaces as duplicate traffic (visible to the operator) rather than silent message loss.

- Human-user messages are KEPT — `RawSenderIdBot == false` short-circuits to KEEP without even comparing identities.

§6.3 Per-channel identity kinds

Adapter	IdentityKind	Source	Example Value
Telegram	IdentityUsername	<code>getMe → result.username</code>	<code>herald_bot</code> (no @)
Slack	IdentityUserID	<code>auth.test → user_id</code>	<code>U01ABCDEF</code>
Email (Wave 8)	IdentityAddress	<code>configured From-address</code>	<code>herald@example.com</code>

A new channel chooses the `IdentityKind` that the channel's native event API exposes on every inbound event; if the channel has no native sender identity (a truly anonymous channel), the channel cannot host bidirectional bot traffic at all — that constraint is upstream of Herald.

§6A. Participant identity, attribution & @-tagging (per `docs/design/PARTICIPANT_ATTRIBUTION.md`)

Beyond the self-filter's bot-identity check (§6, which only answers "is this message from MY own bot?"), Herald relates every message to a **Participant** — a logical Subscriber/User. The same logical person may have a DIFFERENT username on every messenger, so identity is resolved per channel. This is the contract in [docs/design/PARTICIPANT_ATTRIBUTION.md](https://docs.waveprotocol.com/design/PARTICIPANT_ATTRIBUTION.md), inherited from HelixConstitution per §11.4.35.

§6A.1 Per-channel username mapping

A Participant is backed by two PG tables:

- `subscribers` — the logical party: canonical messenger-neutral handle, `display_name, kind ∈ {human, agent, service}`.
- `subscriber_aliases` — the per-channel handle: `subscriber_id, channel, channel_user_id, +NEW username TEXT` (the per-channel @handle used for @-tagging — distinct from `channel_user_id`, which is the chat/user id). `UNIQUE (channel, channel_user_id)`.

The **canonical handle** (the messenger-neutral string stored in a workable item's `created_by/assigned_to`) is either `Claude` (the reserved system-agent sentinel — `kind=agent`, NEVER tagged) or a human's canonical handle (defaults to their Telegram @username since Telegram is the primary messenger). Resolution bridges the two worlds:

- inbound — `ResolveSender(channel, channel_user_id, username)` maps a received message's sender to a canonical handle;
- outbound — `UsernameFor(handle, channel)` returns the @username for a canonical handle on a target channel, reporting not-found when the participant has no alias there (you cannot tag someone who is not on that messenger).

§6A.2 Operator env var

The **operator** — the one human who drives the system via the Claude Code CLI — is designated by env var, NOT a DB flag:

Env var	Example value	Meaning
HERALD_TGRAM_OPERATOR_USERNAME	@milos85vasic	the operator's Telegram @username (primary messenger)
HERALD_<CHANNEL>_OPERATOR_USERNAME	HERALD_SLACK_OPERATOR_USERNAME=...	per-messenger generalization for any other channel

The operator's canonical handle equals this env value (`IdentityResolver.OperatorHandle()`).

§6A.3 @-tagging behaviour

On any workable-item event, the outbound notification to each channel @-tags the participant(s) who must be aware, resolved to that channel's @username:

- tag `assigned_to` when it is a human handle AND `assigned_to != Operator`;
- tag `created_by` when it is a human handle AND `created_by != Operator` AND `created_by != "Claude"`;
- "Claude" is NEVER tagged (it is the system); the Operator is NEVER tagged (no self-ping);
- de-dup, then resolve each handle to the channel's @username and skip any participant with no alias on that channel.

The tgram adapter renders a mention as @username (a Telegram username mention reaches a group member); other adapters render their channel's native mention syntax (Slack <@U...>, etc. — future). Attribution columns + the wiring points are documented in [WORKABLE_ITEMS_INTEGRATION.md](#) §3.6–§3.8.

§7. URL schemes

Adapters accept both env-driven `channels.Config` construction (via the registry) AND a URL-form for ad-hoc / test usage. The URL is parsed at construction so the env-driven path goes through the same parser as the test path.

§7.1 Telegram

`tgram://<bot-token>@<bot-username>/<chat-id>`

- <bot-token> — required, the 1234:AAH... token (URL-encoded : is fine).

- `<bot-username>` — informational; carried to the diary annotation. The actual identity is re-read from `getMe` at `Subscribe` time, so this value is NOT trusted for security decisions.
- `<chat-id>` — required, the numeric default destination.

Example: `tgram://1234567890%3AAAH1lab%2D...@herald_bot/-1001234567890`.

§7.2 Slack

`slack://<bot-token>@<workspace>/<channel-id>[?app_token=<xapp-...>]`

- `<bot-token>` — required, the `xoxb-...` token.
- `<workspace>` — informational (mirrors the per-host convention so `op-supplied` config can carry it for diary annotation); NOT required to dial the Slack API.
- `<channel-id>` — required, the `C...` or `G...` channel id.
- `app_token` — required for `Socket Mode (Subscribe)`. Optional for an outbound-only deployment.

Example: `slack://xoxb-1234-5678@acme/C0123456789?app_token=xapp-1-A0123-456-XYZ`.

§7.3 Where the `env` path lives

`pherald listen` does NOT consult URLs from `env` directly. It reads namespaced `env` vars (`HERALD_TGRAM_*`, `HERALD_SLACK_*`) into a `channels.Config` and constructs via the registry. The URL form is used by `qaherald` scenario configs and by hermetic tests that want a single-string fixture.

§8. Adding a new channel

The framework's hot paths (registry resolution, multi-channel fan-in, generalized self-filter, per-channel inbox, reply routing, `qaherald` scenario harness) all work for free once a new adapter satisfies `channels.Channel`. The implementer checklist:

§8.1 Implementer checklist



1. Create the package. New directory `commons_messaging/channels/<yourname>/` with a `<yourname>.go` containing the `Adapter` type. Reference the Slack adapter as a working template (`commons_messaging/channels/slack/`) — it is the post-Wave-7 reference implementation and intentionally smaller than Telegram's substrate.



2. Satisfy the eight methods. Implement `Name`, `Capabilities`, `Send`, `Subscribe`, `HealthCheck` (the §11.0 outbound contract) plus `SendReplyGeneric`, `BotSelfIdentity`, `DownloadAttachment`. A compile-time assertion `var _channels.Channel = (*Adapter)(nil)` at file scope catches signature drift on the first build.



3. Register at `init()`. Add `func init() { channels.Register("yourname", func(cfg channels.Config) (channels.Channel, error) { ... }) }`.

Validate required `cfg` fields in the constructor and return an explicit error on missing creds — never construct an adapter that will fail at first wire crossing (a §107 PASS-bluff).



4. Blank-import. Add `"github.com/vasic-digital/herald/commons_messaging/channels/yourname"` to `pherald/cmd/pherald/listen.go` and (if you want qaherald coverage) to `qaherald/cmd/qaherald/main.go`. The blank import forces `init()` to run at process start so the registry resolves your channel by name.



5. Namespace your env. Pick a stable short token (max, email, discord) and use it as the prefix for every credential: `HERALD_<YOURNAME>_BOT_TOKEN`, `HERALD_<YOURNAME>_CHANNEL_ID`, etc. Add a branch to `perChannelConfig()` in `pherald/cmd/pherald/listen.go` so unknown-env errors are channel-named (e.g. `HERALD_DISCORD_BOT_TOKEN` required (channel discord enabled)).



6. Inbox subdir. No code needed — `channels.WriteContentAddressed(a.Name(), ...)` already picks `~/herald/inbox/<yourname>/`. Just confirm your `Name()` returns the same string you `Register()`ed.



7. Self-filter wiring. In your `Subscribe` handler, call `channels.StampSender(ev.Raw, isBot, kind, value)` on every event you build; use the matching `IdentityKind` for your channel's native identity. The filter logic in `inbound.Dispatcher` then drops self-echoes for free.



8. URL parser. Implement `NewFromURL("yourname://...")` for parity with Telegram/Slack. Round-trip test it.



9. Tests. Hermetic tests using `httptest.NewServer` to mock the channel API (see `slack/*_test.go` and `tgram/*_test.go` for the pattern). At minimum: `TestSend_...`, `TestSendReply_...`, `TestSubscribe_...`, `TestBotSelfIdentity_...`, `TestDownloadAttachment_...`, `TestNewFromURL_...`. Pin token redaction if your channel uses sensitive credentials.



10. qaherald scenario. Add a `MessengerClient` implementation under `qaherald/internal/messenger/<yourname>/` and a builder case so qaherald can drive round-trip QA against your channel. The bidirectional transcript lands under `docs/qa/HRD-NNN-<run-id>/`.



11. e2e + mutation gates. Add positive-evidence invariants to `scripts/e2e_bluff_hunt.sh` and (when a release cycle next runs a mutation gate) paired mutations covering the critical paths (`Send`, `SendReplyGeneric`, `BotSelfIdentity`).



12. Docs. Add a §X. <Channel> section to THIS guide between Telegram (§3) and the troubleshooting section (§9), following the §3/§4 template (required env, obtaining credentials, transport details, capabilities table, security notes).



13. Spec V3 row. Add the channel to `docs/specs/mvp/specification.V3.md` §11 capabilities matrix and §43 catalogue.



14. Issues row. Open the implementing HRD in `docs/Issues.md`; migrate to `docs/Fixed.md` atomically on close per §11.4.19.

§8.2 What you do NOT touch

The following are channel-agnostic and stay untouched when a new channel lands:

- `pherald/cmd/pherald/listen.go` core dispatcher wiring (the only addition is your `blank-import + perChannelConfig` branch).
- `pherald/internal/inbound/dispatcher.go` (the action-routing core).
- `commons_messaging/channels/channel.go` (the interface).
- `commons_messaging/channels/registry.go` (the registry).
- `commons_messaging/channels/selffilter.go` (the comparator).
- `commons_messaging/channels/inbox.go` (the content-addressed write helper).

A working Wave 7 reference for the "minimal new channel" case is the Slack adapter (`commons_messaging/channels/slack/`) — ~600 lines including tests, all conforming to the checklist above.

§9. Troubleshooting

§9.1 Echo loop after the first few replies

Symptom. The bot replies once; then a second, third, fourth reply arrives within seconds; the long-poll never quiesces.

Root cause. `BotSelfIdentity` returned an empty `Value` (or the wrong identity), so `IsSelfEcho` does not match the bot's own messages and they re-trigger Claude Code.

Fix.

1. Confirm the adapter's `Subscribe` constructor refused to boot on empty identity — if it did boot, the fail-loud invariant was broken; file a bug.
2. Check the identity API result manually:
 - Telegram: `curl -s "https://api.telegram.org/bot<TOKEN>/getMe" | jq '.result.username'` — must be non-empty.
 - Slack: `curl -s -H "Authorization: Bearer <xoxb-...>" https://slack.com/api/auth.test | jq '.user_id'` — must start with U.
3. Confirm the adapter's `StampSender` call carries the same `IdentityKind` the bot self-identity reports (a Telegram adapter that stamps `IdentityUserID` instead of `IdentityUsername` would not match).

§9.2 Slack channel_not_found

Symptom. `chat.postMessage` returns `{"ok":false,"error":"channel_not_found"}`.

Root cause.

1. `HERALD_SLACK_CHANNEL_ID` is a channel name (`#general`) rather than the channel id (`C.../G...`). Names are NOT accepted by the Web API.

2. The bot has not been invited to the channel — run `/invite @your-bot-name` in the target channel from a workspace member account.
3. The bot lacks the right history scope — `channels:history` for public, `groups:history` for private. Re-install the app after adding scopes.

§9.3 Slack `invalid_auth` or `token_expired`

Symptom. Every Web API call returns `{"ok":false,"error":"invalid_auth"}` or `token_expired`.

Root cause. The `xoxb-...` token has been revoked, the workspace's owner rotated app credentials, or the app was uninstalled and reinstalled (re-installation issues a new token).

Fix. Rotate `HERALD_SLACK_BOT_TOKEN` to the freshly-issued value from **OAuth & Permissions** → **Bot User OAuth Token**. Restart `pherald listen`.

§9.4 Slack `not_allowed_token_type` for `apps.connections.open`

Symptom. Socket Mode WebSocket dial fails immediately.

Root cause. `HERALD_SLACK_APP_TOKEN` is empty, or carries the bot token by mistake. Socket Mode connect requires the `xapp-...` app-level token, NOT the `xoxb-...` bot token.

Fix. Confirm `HERALD_SLACK_APP_TOKEN` starts with `xapp-`. If not, regenerate it from **Basic Information** → **App-Level Tokens** with the `connections:write` scope.

§9.5 Attachments arriving in the wrong inbox subdir

Symptom. A Slack photo lands under `~/ .herald/inbox/tgram/` (or vice-versa).

Root cause. Either `HERALD_INBOX_DIR` is set to a path that overlaps another deployment, OR the adapter's `Name()` does not match its registered name (a refactor regression).

Fix.

1. Check `HERALD_INBOX_DIR` for an unintended override.
2. Confirm the adapter's `Name()` method returns the same string passed to `channels.Register()` at `init()` time — they MUST match or the inbox helper picks the wrong subdir.
3. Grep your code for `channels.WriteContentAddressed("<literal-string>", ...)` — a hard-coded literal that drifts from `Name()` is a §107 hazard.

§9.6 `HERALD_CHANNELS=foo` failing at boot

Symptom. `pherald listen: build "foo" channel: channels: unknown channel: "foo" (registered: [slack tgram])`.

Root cause. Typo, or the adapter's package is not blank-imported by `pherald/cmd/` `pherald/listen.go` (so its `init()` never ran).

Fix.

1. Check spelling — compare against `channels.Names()` output (embedded in the error).

2. Confirm the blank import: `grep -E '_ "github.com/vasic-digital/herald/commons_messaging/channels/' pherald/cmd/pherald/listen.go`. Every channel you expect to be available MUST have a matching blank import.

§9.7 Privacy-mode Telegram: bot sees no group messages

Symptom. Telegram DMs work; group messages never arrive.

Root cause. Telegram bots default to privacy-mode-on, which means they only see commands (/foo) and replies + mentions in groups.

Fix. DM @BotFather, /setprivacy, select your bot, choose Disable. Restart pherald listen. The bot will now see every message in groups it is a member of.

§9.8 Telegram Conflict: terminated by other getUpdates request

Symptom. Long-poll boots, then aborts with Conflict.

Root cause. Another process is calling getUpdates against the same token — Telegram permits only one consumer per token. Usually a second pherald listen instance, a forgotten dev shell, or a webhook configured against the same token.

Fix. Kill the competing consumer (`kill -f 'pherald listen'`). If a webhook is the culprit, delete it: `curl -X POST https://api.telegram.org/bot<TOKEN>/deleteWebhook`.

§9.9 Inbox files are zero-byte / .part residue

Symptom. ~/ .herald/inbox/<channel>/dl-*.part files left behind, or content-addressed final files are zero-byte.

Root cause. A streaming download was interrupted (network drop, process kill) before the atomic rename completed; OR a bug in a new adapter's DownloadAttachment returned success before fully writing.

Fix.

1. Delete the .part files — they are deliberate temp residue, the next download will start fresh and idempotently land the final file.
2. If zero-byte FINAL files appear, the new adapter has a bug. Re-check that it streams the body through `io.MultiWriter(file, hasher)` and ONLY returns the path after `os.Rename` succeeds.

§9.10 Cross-channel "wrong reply destination"

Symptom. A reply intended for Slack arrives in Telegram (or vice-versa).

Root cause. The dispatcher's channelRouter keys replies by `recipient.Channel` — if an adapter constructed a `commons.Recipient` with the wrong Channel value, the router sends the reply via the wrong adapter.

Fix. In your adapter's Subscribe handler, confirm every `commons.Recipient` is constructed with `Channel: a.Name()` (the adapter's own name). The `channelRouter` test fixture (`pherald/cmd/pherald/listen_test.go`) pins this for the two live channels; a third adapter must add a parallel assertion.

§10. Audit checklist (operator pre-deploy)

Run through the checklist before every fresh deploy or after every credential rotation.

§10.1 Environment

- ☐ Determine the intended channel set; export `HERALD_CHANNELS=<comma-list>` (or leave unset for Telegram-only).
- ☐ For every channel in the set, every required env var listed in the channel's section is exported.
- ☐ Token prefixes match the channel's expected format:
 - ☐ Telegram token: numeric id, colon, alphanumeric — e.g. `1234567890:AAH...`
 - ☐ Slack bot token: starts `xoxb-`.
 - ☐ Slack app token: starts `xapp-` (only if Slack is in `HERALD_CHANNELS`).
- ☐ Slack channel id: starts `C` (public) or `G` (private), NOT a `#name`.
- ☐ Telegram chat id: numeric (negative for groups, `-100...` for supergroups, positive for DMs).
- ☐ No credential appears in `git ls-files | xargs grep -l '<your-token>'` (manual spot-check before commit).

§10.2 Bot setup

- ☐ **Telegram** — privacy-mode disabled (`/setprivacy → Disable` in BotFather) IF the bot is meant to see group messages.
- ☐ **Telegram** — no other consumer is calling `getUpdates` against the same token. `curl -s 'https://api.telegram.org/bot<TOKEN>/getWebhookInfo' | jq` confirms no stale webhook is configured.
- ☐ **Slack** — bot is a member of the target channel: `/invite @your-bot-name` in Slack.
- ☐ **Slack** — Socket Mode is enabled in **App settings → Socket Mode**.
- ☐

Slack — Bot Token Scopes include all of: `chat:write`, `files:read`, `files:write`, `channels:history` (and/or `groups:history`).



Slack — App-Level Token scope: `connections:write`.

§10.3 Registry resolution



Confirm every enabled channel is registered. From the repo root: `grep 'channels.Register' commons_messaging/channels/**/*.go` lists every available channel. The set **MUST** contain every name listed in `HERALD_CHANNELS`.



(Future) A `pherald listen --dry-run` (planned — file as a follow-up HRD if needed) prints `channels.Names()` + the resolved per-channel config without crossing the wire.

§10.4 Filesystem



`~/.herald/inbox/` exists with mode `0700` (or `$HERALD_INBOX_DIR`, if overridden).



Per-channel subdirs are present for every enabled channel after the first inbound attachment arrives — `ls -la ~/.herald/inbox/`.



Disk has headroom for the worst-case attachment burst (`AttachmentMaxMiB` × expected concurrent uploads).

§10.5 §107.x evidence



Plan the run-id directory for this session: `docs/qa/HRD-<NNN>-<TS>Z/`.



If using `qaherald`, point its `--qa-out-dir` at that path and confirm a transcript / attachments tree is captured before tagging.



For Telegram-driven manual runs, screenshot both directions of the conversation and commit the screenshots in the run-id directory.



For Slack-driven manual runs, save the conversation transcript (Slack right-click → **Copy link**, or the App Home transcript export) and commit it.

§10.6 First-message smoke



Start `pherald listen` in the foreground; observe the boot lines for each enabled channel:



`pherald listen: starting Telegram getUpdates long-poll loop`

- ☐ (Slack equivalent — Socket Mode WebSocket connected).
 - ☐ Send a test message from a human user. Confirm:
 - ☐ The bot replies (Claude Code dispatch round-trip).
 - ☐ The reply quotes the original message (`reply_to_message_id / thread_ts`).
 - ☐ The conversation appears in `docs/herald/diary/main.md`.
 - ☐ Send a test message FROM the bot to itself (loopback). Confirm the self-filter drops the echo — no second reply arrives, no Claude Code dispatch in the logs.
-

§11. References

§11.1 Source files

- `commons_messaging/channels/channel.go` — the eight-method interface + `SelfIdentity` types.
- `commons_messaging/channels/registry.go` — `Register / New / Names + ErrUnknownChannel`.
- `commons_messaging/channels/inbox.go` — `InboxDir, WriteContentAddressed, MimeToExt`.
- `commons_messaging/channels/selffilter.go` — `StampSender, IsSelfEcho`.
- `commons_messaging/channels/tgram/*` — Telegram adapter (Wave 6 substrate).
- `commons_messaging/channels/slack/*` — Slack adapter (Wave 7 T6 reference implementation).
- `pherald/cmd/pherald/listen.go` — `loadEnabledChannels, perChannelConfig, channelRouter, runListen`.

§11.2 Spec V3

- §11 — Channel capabilities matrix (every channel's row).
- §11.0 — outbound `commons.Channel` contract (the five embedded methods).
- §32.2 — inbound long-poll / Socket Mode requirements.
- §32.9 — anti-echo-loop self-filter mandate.
- §43 — Channel + flavor catalogue.
- §107.x — `docs/qa/<run-id>/` evidence mandate (Helix §11.4.83 cascade).

§11.3 HRDs

HRD	Wave	Task	Description
HRD-110	7	T1	Extract <code>channels.Channel</code> interface (richer inbound contract embedding §11.0).
HRD-111	7	T2	<code>init()</code> -based channel registry + <code>channels.New / channels.Names</code> .

HRD	Wave	Task	Description
HRD-112	7	T3	Per-channel inbox subdirs (~/.herald/inbox/<channel>/).
HRD-113	7	T4	Generalize bot self-filter via BotSelfIdentity + IsSelfEcho.
HRD-114	7	T5	Multi-channel pherald listen (fan-in + channelRouter).
HRD-115	7	T6	Slack channel adapter (Socket Mode + Web API).
HRD-116	7	T7	qaherald Slack MessengerClient + scenario builder.
HRD-117	7	T8	Spec V3 §11.0 + §32.2 + §43 update.
HRD-118	7	T9	scripts/e2e_bluff_hunt.sh multi-channel invariants (E81..E88).
HRD-119	7	T10	Wave 7 paired §1.1 mutation gate (tests/test_wave7_mutation_meta.sh).
HRD-120	7	T11	§11.4.85 stress + chaos for the multi-channel runtime.
HRD-121	7	T12	This guide + Issues→Fixed migration + v0.6.0 release prep.

§11.4 Related guides

- [OPERATOR_CREDENTIALS.md](#) — umbrella credentials guide; defines the .env resolution order and the dual-source model (shell exports vs .env).
- [messengers/TELEGRAM.md](#) — Telegram-specific deep-dive (HRD-011).
- [messengers/SLACK.md](#) — Slack-specific deep-dive (HRD-115; landing alongside this guide).
- [HERALD_CONSTITUTION.md](#) — §107 end-user-usability covenant + §107.x evidence mandate.
- [CONSTITUTION_INHERITANCE.md](#) — parent-discovery rules.

§11.5 Wave 7 plan

- docs/superpowers/plans/2026-05-27-wave7-generic-messenger.md — the full Wave 7 plan (Tasks 1..12, locked decisions, file structure).

§11.6 Constitutional anchors

- Helix §11.0 — outbound Channel contract.
- Helix §11.4 / §11.4.5 / §11.4.6 — anti-bluff captured-evidence mandate.
- Helix §11.4.83 — docs/qa/<run-id>/ evidence cascade (Herald §107.x).
- Helix §11.4.85 — stress + chaos test mandate (T11 binding).
- Helix §11.4.94 — zero-idle parallel-by-default operating mode.
- Helix §11.4.96 — safe-parallel-work catalogue.

Sources verified

Per HelixConstitution §11.4.99 + Herald §108.n (Latest-Source Documentation Cross-Reference Mandate). Every operator-facing instruction in this document was cross-referenced against the LATEST official online documentation of the relevant service before publication.

Last verified: 2026-05-28

Source	URL / path	Authored / verified
	https://core.telegram.org/bots/api	

Source	URL / path	Authored / verified
Telegram official Bot API documentation		§3 (HERALD_TGRAM_BOT_TOKEN format from BotFather; HERALD_TGRAM_CHAT_ID per-chat-type shape — private positive / group negative / supergroup -100...); §3.2 (/newbot username-ending-in-bot rule, /setprivacy Disable flow); §3.3 (chat-id discovery via getUpdates); §3.4 (sendMessage / sendPhoto / sendDocument / sendVoice / MarkdownV2 wire); §3.4 (long-poll 25 s timeout + getUpdates semantics); §3.5 (AttachmentMaxMiB = 50 hard cap); §9.7 + §9.8 (privacy-mode, Conflict: terminated by other getUpdates request).
Telegram Bot Features — Privacy Mode	https://core.telegram.org/bots/features#privacy-mode	§3.2 step 4 (/setprivacy Disable); §9.7 (the "privacy-mode-on is the #1 stuck-cause" guidance).
Slack Web API method index	https://api.slack.com/methods	§4 (Slack adapter overview — chat.postMessage, files.uploadV2, auth.test, conversations.history); §4.5 (xoxb- token redaction rationale — workspace-scope credential semantics); §9.2 (channel_not_found); §9.3 (invalid_auth / token_expired).
Slack Events API + Socket Mode	https://api.slack.com/apis/connections/socket + https://api.slack.com/apis/events-api	§4.2 (Socket Mode WebSocket setup — connections:write scope on app-level token; xapp- prefix; Enable Socket Mode toggle; apps.connections.open dial); §4.3 (Socket Mode == no inbound NAT traversal / no public URL); §9.4 (not_allowed_token_type for apps.connections.open).
Slack OAuth + Bot Token Scopes	https://api.slack.com/scopes	§4.2 step 3 (chat:write / files:read / files:write / channels:history / groups:history / users:read Bot Token Scopes); §4.2 step 6 (xoxb- Bot User OAuth Token install); §10.2 audit-checklist entries.
Slack reply threading (thread_ts)	https://api.slack.com/messaging/retrieving#threads	§4.3 (thread_ts as Slack's reply anchor; per-channel router's reply-routing key).
telebot.v3 vendored library	submodules/telebot/ (v3.3.8, pinned per §11.4.74)	§1.4 (init()-registered tgram constructor); §6.3 (bot.Me.Username → IdentityUsername).
slack-go vendored library	submodules/slack-go/	

Source	URL / path	Authored / verified
Empirical Herald operator testing 2026-05-28	docs/qa/HRD- LIVE-20260528T082128Z/	§4 + §9 (Slack adapter behaviour — Web API client, Socket Mode handler, retry semantics). §3 LIVE-since-Wave-6 status; §4 Slack LIVE-since-Wave-7-T6 status; §1.5 multi- channel fan-in semantics.
HelixConstitution §11.4.99 (this document's authority)	<parent>/constitution/ Constitution.md §11.4.99 (HelixConstitution commit c640947)	This footer pattern + 90-day cadence.

Re-verification cadence (per §11.4.99 (C)): Both Telegram and Slack are risk-classified services (per §11.4.99 (D) — bot APIs face anti-abuse / token-revocation; OAuth scope changes can break installed apps silently). Telegram-specific subsections (§3, §9.7, §9.8) → **90-day max staleness**, next due **2026-08-26**. Slack-specific subsections (§4, §9.2..§9.4) → **90-day max staleness**, next due **2026-08-26**. Channel-agnostic substrate (§1, §2, §5, §6, §7, §8, §10) → **180-day max staleness**, next due **2026-11-24**. Re-verify earlier on: Telegram Bot API changelog entry, Slack API changelog entry (<https://api.slack.com/changelog>), operator-error reports, Herald vN.0.0 release boundary.

End of guide.